Multi-objective Young Couple

Problem 1

Setup the full bi-objective model.

Parameters

- $Time_{p,j}$: Time for person p to do job j. This is the same as in the first version, just with 3 more jobs

Decision variables

We only show the new variables:

- y : The variable measuring the absolute time difference between Eve and Steven

$$
\begin{array}{llcr}
\texttt{Minimize.} & \sum_{p,j} Time_{p,j} \cdot x_{p,j} & & \\
 & y & & \\
\texttt{Subject to :} & & & \\
 & \sum_{p} x_{p,j} & = & 1 \;\forall\; j \\
 & y & \geq & \sum_{j} Time_{1,j} \cdot x_{1,j} - \sum_{j} Time_{2,j} \cdot x_{2,j} \\
 & y & \geq & \sum_{j} Time_{2,j} \cdot x_{2,j} - \sum_{j} Time_{1,j} \cdot x_{1,j} \\
 & x_i \in \{0,1\} & y \in R^{+} &
\end{array}
$$

```
#************************************************************************
# Multi-Objective YoungCouple Assignment,
#"Mathematical Programming Modelling" (42112)
#************************************************************************
# Intro definitions
using JuMP
using Gurobi
using HiGHS
using MultiObjectiveAlgorithms
#************************************************************************
```

```
#***********************************************************************
# PARAMETERS
Jobs = ["clean" "cook" "dishwashing" "laundry" "lawn" "shopping" "carwash"]
J=length(Jobs)
Persons = ["Eve", "Steven"]
P=length(Persons)

Time = [ # Time[p,t]
         4.5     7.8      3.6      2.9    1.1     2.1     1.9         2.5    ;
         4.9     7.2      4.3      3.1    2.3     1.5     3.2         1.2
         ]
#***********************************************************************


#***********************************************************************
# Model
YoungCouple2 = Model(Gurobi.Optimizer)

@variable(YoungCouple2, x[1:P, 1:J], Bin) # 1 if person p performs job j

@variable(YoungCouple2, 0 <= y) # diff in working time

# Minimize work-time
#@objective(YoungCouple2, Min,
#           sum(Time[p,j]*x[p,j] for p=1:P, j=1:J))

@expression(YoungCouple2, time_expr, sum(Time[p,j]*x[p,j] for p=1:P, j=1:J))
@expression(YoungCouple2, dist_expr, y)
# Define combined BI-OBJECTIVE
@objective(YoungCouple2, Min, [time_expr,dist_expr])


# each job should be performed
@constraint(YoungCouple2, [j=1:J],
            sum(x[p,j] for p=1:P) == 1)

# force equality measurement variable
@constraint(YoungCouple2, y >= sum(Time[1,j]*x[1,j] for j=1:J) -  sum(Time[2,j]*x[2,j] for j
@constraint(YoungCouple2, y >= sum(Time[2,j]*x[2,j] for j=1:J) - sum(Time[1,j]*x[1,j] for j=
#print(YoungCouple2)
#***********************************************************************


#***********************************************************************
# solve
# Set MIP solver
```

```
set_optimizer(YoungCouple2, () -> MultiObjectiveAlgorithms.Optimizer(HiGHS.Optimizer))
set_silent(YoungCouple2)

# Set MO solver

# EpsilonConstraint, Works
set_attribute(YoungCouple2, MultiObjectiveAlgorithms.Algorithm(), MultiObjectiveAlgorithms.E
optimize!(YoungCouple2)
#println("Termination status: £(termination_status(YoungCouple2))")
#************************************************************************


#************************************************************************
# Solution
if result_count(YoungCouple2)>=1
    println("No Pareto optimal points: ",result_count(YoungCouple2))
    for i in 1:result_count(YoungCouple2)
        obj_vec = objective_value(YoungCouple2; result = i)
        xx=round.(Int,value.([x[p,j] for p=1:P,j=1:J]; result = i))
        eve_work=sum(Time[1,j]*xx[1,j] for j=1:J)
        steven_work=sum(Time[2,j]*xx[2,j] for j=1:J)
        println("Total Work: ",round( obj_vec[1] , digits=1), "    Work difference: ",round(
    end

else
    println("No solutions found")
end
#************************************************************************

#************************************************************************
println("Successfull end of $(PROGRAM_FILE)")
#************************************************************************
```